

Sistema de listas de espera para el servicio gallego de salud implementado en la blockchain de Ethereum

Luca D'angelo Sabin, Luis Gómez Morán

1. Introducción

1.1. Motivación

Para saber la motivación por detrás de esta idea de proyecto es importante conocer el marco en el que nos encontramos actualmente respecto las listas para operaciones de los servicios públicos de salud. Para este caso de estudio se va a utilizar el SERGAS (Servizo Galego de Saúde) como sujeto principal para exponer los principales fallos que pueden tener el sistema actual y demostrar cómo puede ser implementado uno nuevo en su lugar.

La falta de transparencia en las listas para los servicios públicos, comúnmente operaciones y cirugías, siempre ha sido una fuente de críticas por la sospecha de manipulación del SERGAS, sea por beneficiar a personas concretas o engañar las métricas y estadísticas. Es común ver en noticiarios cabeceras tal como las siguientes:



Figura 1: Noticiario de 'elDiario' del 23 de noviembre de 2023. Fuente original.

1.2. Definición del escenario

Se propone implementar el sistema de listas para operaciones en la Blockchain de Ethereum, de manera que la lista sea en todo momento visible para cualquier persona y todos sus cambios queden registrados y justificados de manera inmutable. También se limitará el acceso a las operaciones para solo especialistas del campo de la salud o el personal de gestión de la entidad de salud (SERGAS en este caso) tengan permisos para modificar las listas.

Para una primera implementación básica del sistema se recopiló información acerca del funcionamiento de las listas actuales [1][2], y de momento se omitieron algunos aspectos como

la prioridad y la posibilidad de derivación a convenios privados. La parte correspondiente al análisis de cada caso individual, y la asignación de prioridad continuará recayendo sobre los especialistas de salud y la gestión del SERGAS, con la diferencia de que la información será pública.

El sistema entonces funciona con una cola por cada nivel de prioridad especificado por el SERGAS, con sus operaciones de añadir, borrar, y modificar los pacientes, donde cada entrada de paciente tiene un especialista (dado de alta previamente por el SERGAS) asignado. Todas las operaciones que se realicen en las colas quedan grabadas en 3 historiales distintos, públicos para todos: historial pacientes agregados y retirados de la lista, historial de movimientos en los puestos de la lista, y un historial para las transferencias de especialistas de la lista. Cuando se menciona 'la cola' del contrato se hará referencia a la unión de las 3 colas distintas.

1.3. Objetivos

- Diseñar e implementar un sistema inmutable y transparente para las colas del SERGAS.
- Mejorar la experiencia de los usuarios mediante acceso público y directo a su posición en la cola.
- Aumentar la eficiencia administrativa y reducir errores manuales.

2. Estudio del estado del arte

2.1. Análisis de soluciones parecidas

Actualmente no conseguimos ninguna otra solución existente en el mercado español, sin embargo nos topamos con artículos acerca del tema siendo mencionado. Principalmente nos llamo la atención que dentro de España Javier Colás, director de Innovación en Health Care Institute de Esade y presidente en Additum Blockchain, es portavoz de que la blockchain debería ser incorporado al sistema de sanidad para mejorar la actual atención de pacientes.

Adicionalmente, encontramos un articulo que menciona el como la salud el farma podría incorporar el uso de la blockchain, donde remarcan como ofrece trazabilidad de las cadenas de suministro y los inconvenientes que presentan para las empresas en la actualidad, como puede ser riesgos legislativos, la exposición al riesgo cibernético y el riesgo de ejecución.

Entorno

La sanidad se retrasa y cae a la cola en la implantación del 'blockchain'

Aun así, este conjunto de tecnologías ha podido penetrar en el sector a través de las opciones que ofrece la trazabilidad de las cadenas de suministro. La salud y el **farma** no han desarrollado aplicaciones para cubrir la autenticidad de un producto.



J. Vera | 20 jul 2022 - 04:58



Lo más leído

Médicos autónomos y hospitales se unen para proteger Muface

Tramadol y paracetamol: el matrimonio analgésico que se ha vuelto una mina de oro

Amelia Virtual Care se fusiona con XRHealth para crecer en realidad virtual

Figura 2: Noticiario de 'PlantaDoce' del 20 de Julio de 2022. Fuente original.

Además, investigando conseguimos información acerca de que Estonia implementa desde 2013 blockchain en su sistema de salud como una capa adicional de seguridad para asegurar la integridad de los registros médicos. Donde principalmente se utiliza para almacenar en la blockchain cada uno de los cambios en los registros incluyendo marcas temporales, esto también permite que siempre se utilice la versión más actual del archivo al realizar cambios en este.

Estonia has become the first country to use blockchain for healthcare on a national scale

“We are using blockchain as an additional layer of security to help us ensure the integrity of health records. Privacy and integrity of healthcare information are a top priority for the government and we are happy to work with innovative technologies like the blockchain to make sure our records are kept safe,” said Artur Novek, the foundation’s Implementation Manager and Architect.

It’s not the health records that are secured using blockchain, but the log files that record all the data processing activities performed on those records. Securing such highly private health information from prying eyes is only part of the goal. It is just as important to mitigate every risk that life-critical personal data could become compromised by an unwitting or malicious hacker or a fraudulent insider.

Figura 3: Web de 'Nortal' del 20 de Febrero de 2018. Fuente original.

2.2. Comparativa con la solución propuesta

1. Funcionalidades:

El sistema de Estonia utiliza blockchain para garantizar la inmutabilidad y trazabilidad de los historiales médicos de los ciudadanos, proporcionando acceso controlado a los pacientes y auditando cada interacción con sus datos. Nuestra propuesta para el SERGAS adopta un enfoque similar, pero está específicamente enfocada en la gestión de colas, utilizando contratos inteligentes para registrar posiciones y cambios de prioridad de forma inmutable. Mientras el sistema estonio abarca todo el ecosistema de datos sanitarios, nuestra solución se centra en mejorar la transparencia y eficiencia en la asignación de citas, ofreciendo a los pacientes la posibilidad de auditar los cambios en tiempo real, algo que el sistema de Estonia no implementa para este propósito específico.

2. Ciberseguridad (completar)

3. Análisis

3.1. Actores

- **Entidad responsable (SERGAS):** La entidad que maneja las colas, en este caso el SERGAS.
- **Especialistas:** Un especialista de la salud verificado al que el paciente fue derivado y tuvo su aprobación para solicitar una operación y poner a su paciente en las listas públicas de operaciones.
- **Pacientes:** Cualquier paciente que esté en las colas.

- **Externos/Audidores:** Cualquier usuario, sea partícipe o ajeno al contrato. No tienen por qué estar relacionados con la entidad responsable o especialistas.

3.2. Requisitos

- Requisitos funcionales:
 - El sistema debe permitir registrar, editar y eliminar pacientes.
 - Todas las operaciones deben de quedar guardadas en un historial inmutable, visible para todos.
 - Se debe de poder rastrear en todo momento cada paciente y/o especialista.
- Requisitos no funcionales:
 - La aplicación debe seguir funcionando ante retrasos o delays en las operaciones de la blockchain.
 - Solamente usuarios autenticados dentro del contrato podrán modificar sus datos o visualizar datos privados.
- Requisitos técnicos: Debe de tener su funcionalidad implementada en la blockchain de Ethereum e implementar un caso de uso con IPFS.
- Requisitos legales: debe cumplir con el reglamento GDPR y garantizar la confidencialidad de información médica y privada.

3.3. Justificación del uso de Ethereum

- Soporte para smart contracts, lo cual permite implementar código en la blockchain.
- Inmutabilidad: Ethereum garantiza que los registros almacenados en la blockchain no puedan ser alterados una vez creados. Esto asegura que las posiciones en las colas y cualquier cambio en ellas sean permanentemente trazables y transparentes, eliminando la posibilidad de manipulaciones internas o externas.
- Accesibilidad: Ethereum es una red pública, lo que permite la compatibilidad con herramientas ampliamente utilizadas para la auditoría y verificación de datos. Esto facilita que los pacientes y auditores puedan acceder y revisar la información sin depender de sistemas propietarios.
- Descentralización: Al operar sobre una red descentralizada, Ethereum elimina la dependencia de un único punto de fallo. Esto asegura la estabilidad y el funcionamiento continuo del sistema, incluso en caso de fallos técnicos o intentos de sabotaje dirigidos a infraestructuras centralizadas.

4. Diseño

4.1. Casos de uso

En el sistema planteado tenemos distintos casos de uso para cada actor que interactuará con el contrato:

- **Entidad responsable:** Responsable por el manejo de las colas. Internamente se denomina `OWNER_ENTITY`.
- **Especialista:** Dan de alta a pacientes en las colas. Internamente denominado `SPECIALIST`.
- **Paciente:** Podrá ver su posición concreta en la lista en cualquier momento. Internamente denominado `PATIENT`.
- **Externo/Audidores:** Podrá consultar las listas y todos los historiales de modificaciones que sufrió. También podrá ver los pacientes y especialistas que se encuentran en las listas.

Todos los roles tienen los casos de uso de los usuarios externos.

Las funcionalidades de **OWNER_ENTITY** son:

- `addSpecialist(dir_paciente, nombre)`: Le asigna el rol de `SPECIALIST` a una dirección.
 - Devuelve el especialista creado.
- `removeSpecialist(dir_especialista)`: revoca el rol de `SPECIALIST` a una dirección.
- `changePriority(dir_paciente, prioridad, nueva_prioridad, comentario)`: Cambia a un paciente de una cola de prioridad a otra, añadiéndolo al final de la nueva cola.
- `movePatientInQueue(dir_paciente, nueva_posición, comentario)`: Cambia la posición de un paciente dentro de su cola actual.
- `attendPatient(dir_paciente, comentario)`: Marca a un paciente específico como atendido y lo elimina de la cola correspondiente.
- `attendPatientAtPosition(prioridad, índice, comentario)`: Marca como atendido al paciente en una posición específica de una cola.
- `derivePatient(dir_paciente, comentario)`: Marca a un paciente como derivado, eliminándolo de la cola correspondiente.
- `transferAllPatientsFromSpecialist(dir_especialista_antiguo, dir_nuevo_especialista, comentario)`: Transfiere todos los pacientes de un especialista a otro.
- `getCompletePatientData(hash_de_paciente)`: Función para obtener y deanonimizar los datos de un paciente utilizando su identificador público hashado.
 - Devuelve datos completos del paciente.

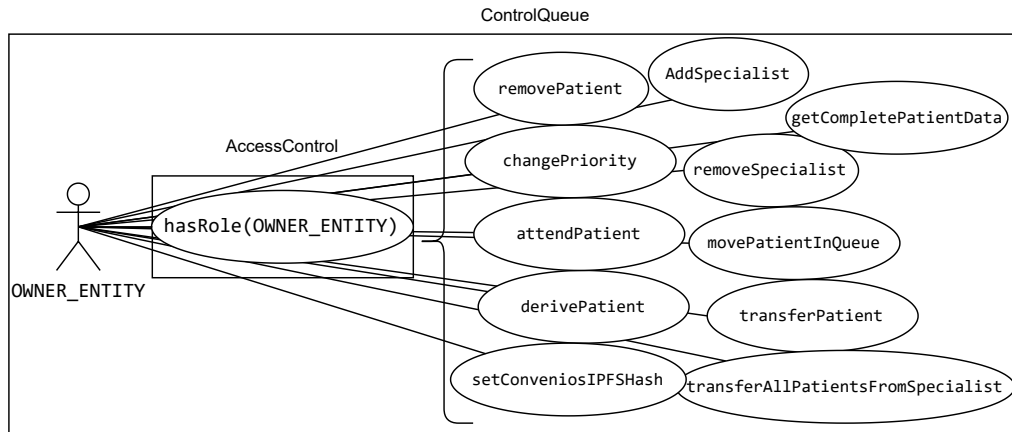


Figura 4: Diagrama de casos de uso de la entidad dueña del contrato.

Las funcionalidades de **SPECIALIST_ROLE** son:

- `addPatient(dir_paciente, nombre, prioridad, comentario)`: Añade un paciente a la cola correspondiente según su prioridad y le asigna al especialista actual.
 - Devuelve el objeto de paciente creado con su información completa.
- `transferPatient(dir_paciente, dir_especialista_nuevo, comentario)`: Transfiere un paciente a otro especialista.
- `transferAllPatients(dir_especialista_nuevo, comentario)`: Transfiere todos los pacientes asignados al especialista actual hacia otro especialista.
- `removePatient(dir_paciente, comentario)`: Elimina a un paciente de la cola. Verifica que el paciente pertenece al especialista antes de realizar la acción.
- `getPositionInQueue(dir_paciente)`: Se usa para obtener la información del paciente y su posición actual en la cola correspondiente.
 - Devuelve una tupla con el objeto de paciente anónimo y su posición en la cola.
- `getCompletePatientData(hash_de_paciente)`: Función para obtener y deanonimizar los datos de un paciente utilizando su identificador público hasheado.
 - Devuelve datos completos del paciente.

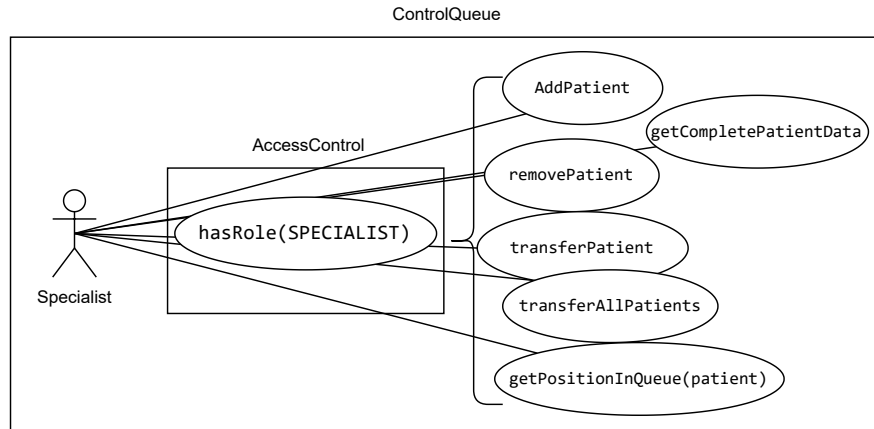


Figura 5: Diagrama de casos de uso para el especialista.

Las funcionalidades de **PATIENT_ROLE** son:

- `getPositionInQueue()`: Devuelve la información del paciente que realiza la llamada, incluyendo su posición actual en la cola correspondiente.

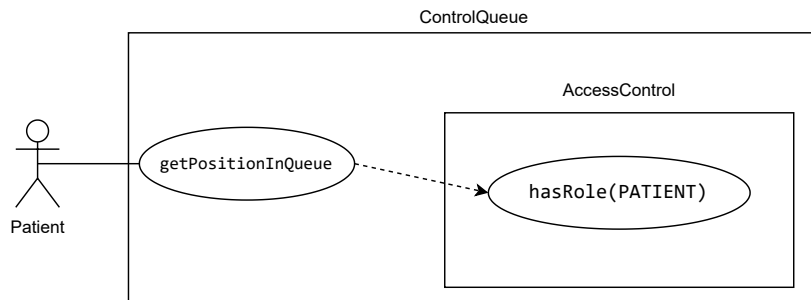


Figura 6: Diagrama de casos de uso para el paciente.

Las funcionalidades de los agentes externos (PUBLIC) son:

- `getQueueType()`: Devuelve el nombre del contrato para qué tipo de cola está asignado.
- `patientBelongsToSpecialist(dir_paciente, dir_especialista)`: Verifica si un paciente pertenece a un especialista determinado.
 - Devuelve un booleano.
- `getSpecialists()`: Devuelve la lista completa de especialistas registrados.
 - Devuelve una lista de objetos Especialista.
- `getPatientsFromSpecialist(dir_especialista)`: Devuelve una lista de todos los pacientes asignados a un especialista específico.
 - Devuelve una lista de objetos de Paciente anónimo.
- `getOwnerEntityAddress()`: Devuelve un string con la dirección del propietario de la entidad (OWNER_ENTITY).

- `getMovementHistory()`: Devuelve el historial de movimientos realizados en la cola, incluyendo cambios de posición o de prioridad.
- `getAddRemoveHistory()`: Devuelve el historial de adiciones y eliminaciones de pacientes en la cola.
- `getUpdateHistory()`: Devuelve el historial de actualizaciones de especialistas relacionadas con los pacientes.
- `getQueue(prioridad)`: Obtiene la lista de pacientes de la cola correspondiente a una prioridad específica.
 - Devuelve una lista de objetos de Paciente anónimo.
- `getQueueLength(prioridad)`: Devuelve un int con la longitud actual de la cola de una prioridad específica.
- `getRole()`: Devuelve un número de 4 bits, donde cada bit representa un rol del contrato. Del más significativo al menos son: OWNER_ENTITY_ROLE, SPECIALIST_ROLE, PATIENT_ROLE y PUBLIC. De esa manera se puede tener varios roles por usuario y fácilmente saber cuáles son.

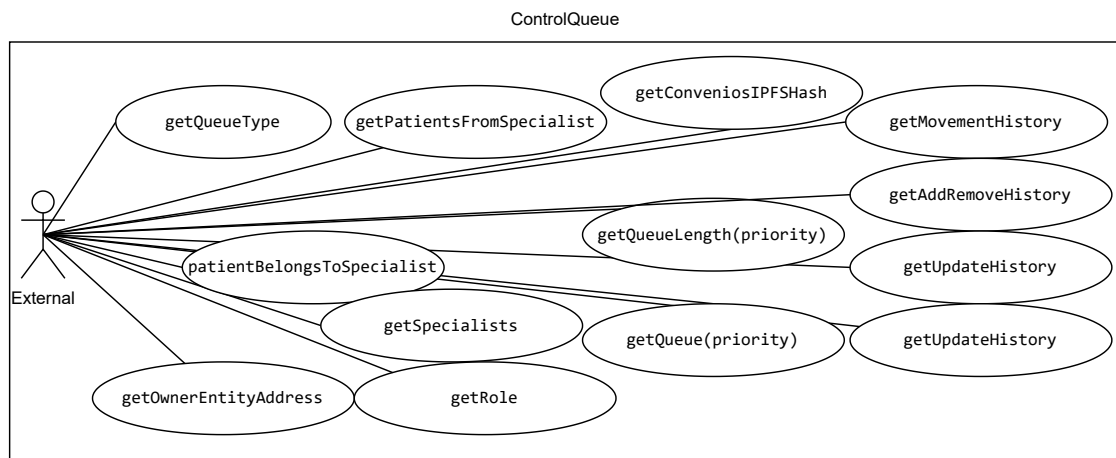


Figura 7: Diagrama de casos de uso para usuarios externos.

4.2. Diagramas de secuencia

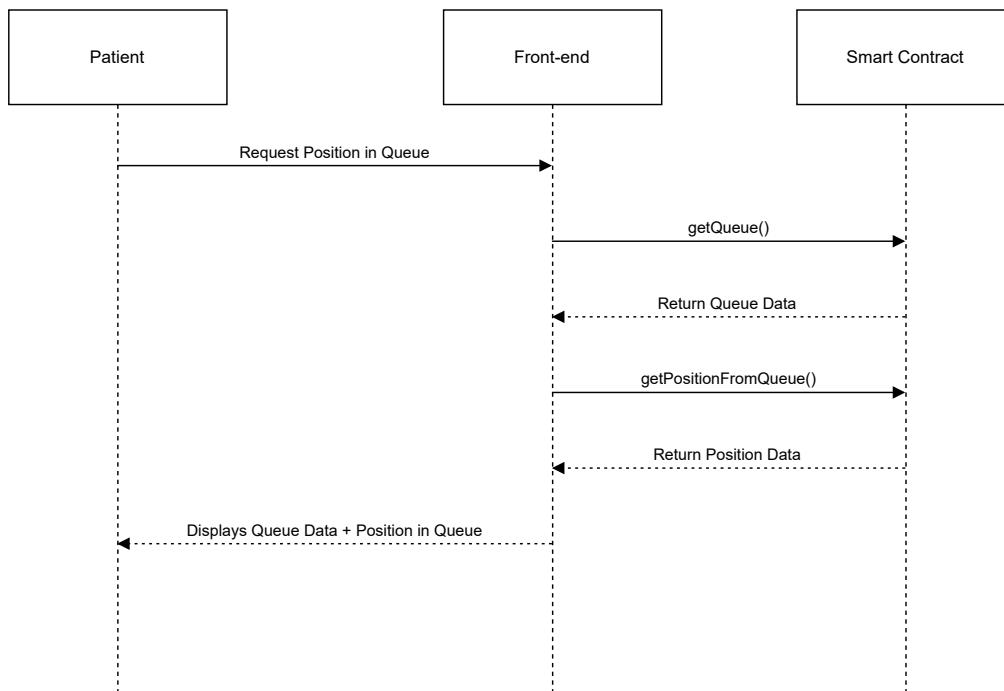


Figura 8: Diagrama de secuencia para un Paciente obteniendo su posición en la cola.

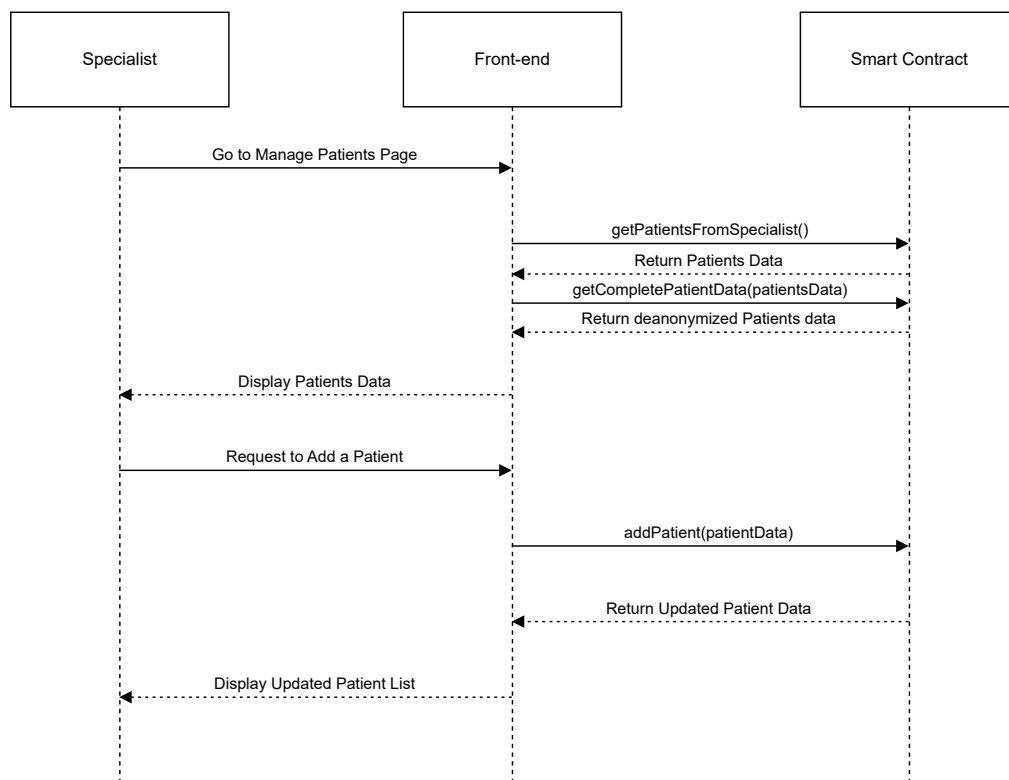


Figura 9: Diagrama de secuencia para un especialista añadiendo a un paciente a una cola.

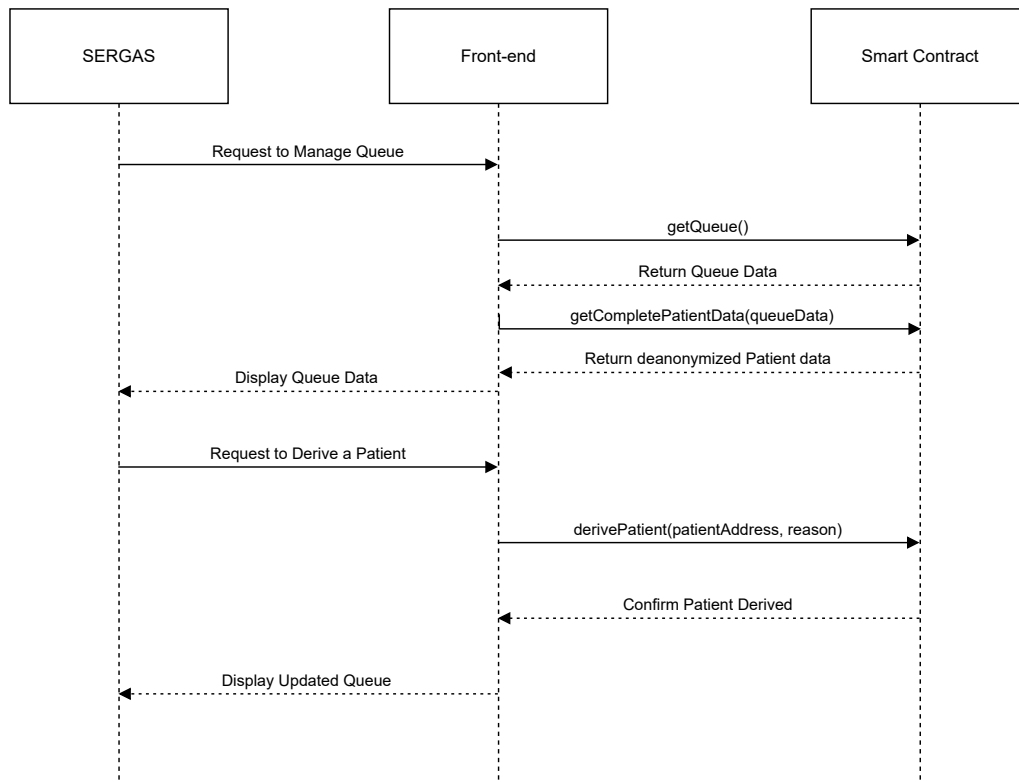


Figura 10: Diagrama de secuencia para el SERGAS derivando un paciente en la cola.

4.3. Arquitectura de comunicaciones

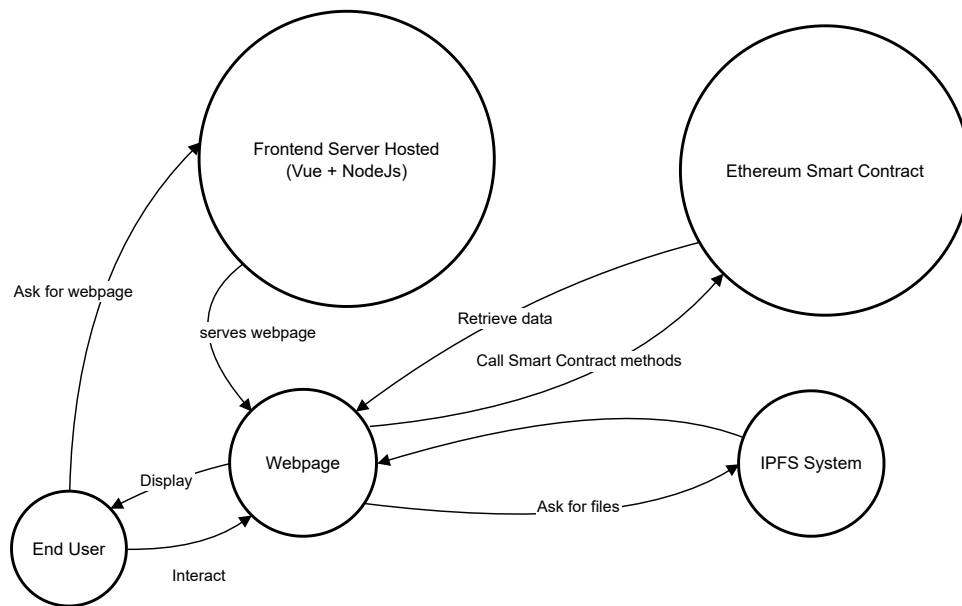


Figura 11: Arquitectura de comunicaciones. Donde el usuario accede a un servidor remoto con el frontend y obtiene la página web. Desde el cliente del usuario, se realizan todas las peticiones y operaciones al smart contract y el IPFS.

4.4. Modelo de datos

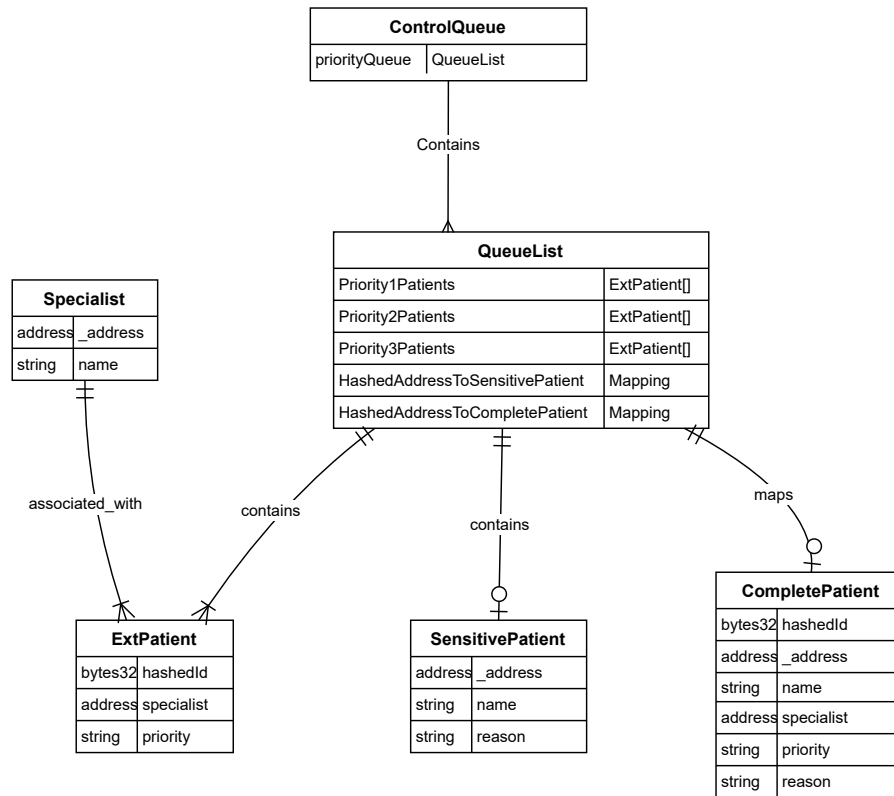


Figura 12: Arquitectura de datos de QueueList, que es el módulo principal de manejo de colas dentro del contrato (ControlQueue).

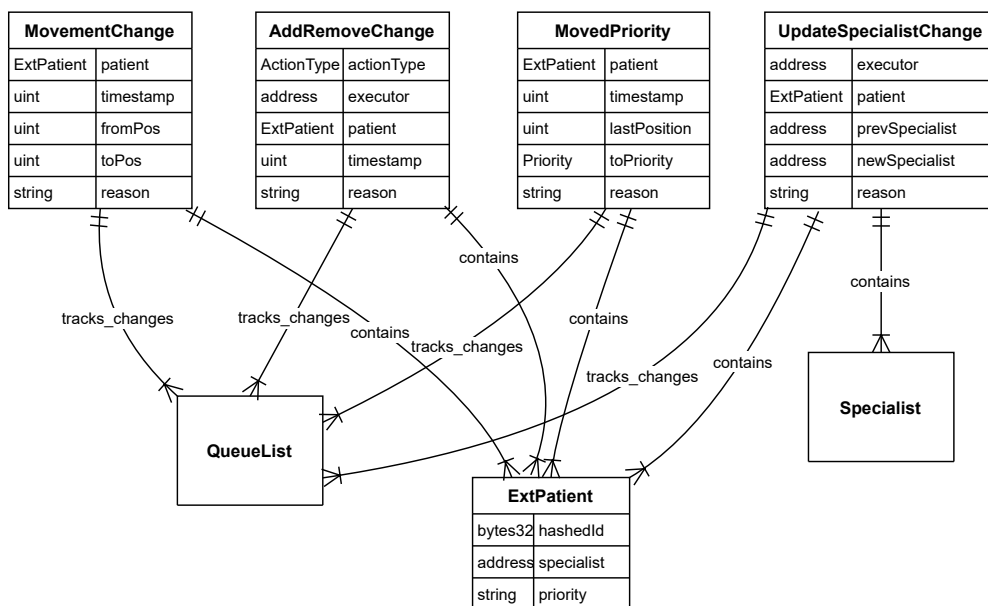


Figura 13: Arquitectura de datos y relaciones del historial de modificaciones dentro de QueueList.

4.5. Tecnologías utilizadas

El smart contract se implementó en **Solidity**, y fue desplegado en una testnet de Ethereum llamada Sepolia. Para el front-end se utilizó **Vue**, un framework de Javascript para crear interfaces Web de usuario, para hacer una SPA que se ejecuta con un runtime de NodeJS y vite para compilar y servir. En lugar de Javascript, se usó TypeScript para tipado estático.

Para la interacción del contrato y el wallet se usa la librería de **Web3JS**. Está pensada para la interacción el un wallet de **MetaMask**, ya que es el más sencillo y de uso extendido. También se utiliza IPFS para la subida y recuperación archivos.

En la aplicación se usa **Pinia** para las stores, donde se guardan métodos e información que se comparte en toda la aplicación y que sean de fácil acceso. Se utiliza también la librería de Tailwind CSS para facilitar la implementación del CSS en el front-end. Para algunos iconos también se usa la librería de Fonts awesome.

5. Implementación

El funcionamiento de las colas esta inspirado, en la medida de lo posible, en como funcionan en el SERGAS. Existen 3 niveles de prioridad en las colas para operaciones [3]. El proceso de asignación de prioridad y posición de la cola se hace caso por caso de manera individual, llevando en cuenta una variedad de factores: como afecta en la vida cotidiana de la persona, si se puede permitir esperar a que se haga la operación, si se puede agravar con el tiempo, estado actual de la cola, etc. Por ello, el proceso de decisión en a posición de la cola se continua delegando a la entidad de manera externa al contrato. Con todo, se ofrecen campos de 'razón' o 'comentarios' para cada operación que se realiza en la cola para plasmar qué llevó a la toma de decisión.

5.1. Colas

Se considera que el caso de uso más abundante será *getPositionInQueue*, ya que son múltiples usuarios los que lanzarían esa consulta. Por lo tanto, la estructura de datos que implemente la cola procurará optimizar la búsqueda de un elemento antes que la modificación y el borrado. En ciertas implementaciones la obtención de la lista ordenada también puede ser más costosa, por lo que en un futuro se podría plantear la eliminación del caso de uso de *getQueue*, si eso hiciese que el resto de operaciones fuesen más eficientes.

Para la implementación de las colas se contemplaron las siguientes alternativas:

- **Priority queue (con un Max Heap):** A pesar de parecer la implementación más adecuada, trae consigo varias desventajas para el caso planteado: No hay una jerarquía clara sobre la que organizar el árbol, ya que hay pocos niveles de prioridad y las operaciones de recuperación de valores de la cola no son dependientes de ninguna jerarquía en concreto. También existe una necesidad de flexibilidad sobre los elementos de una misma prioridad, ya que cada caso se analizaría individualmente. Aunque sí es cierto que implementa las operaciones de modificación inserción (incluso si es al final) y borrado en $O(\log n)$, no tenemos una propiedad sobre la que ordenarlo ni tampoco la necesidad de acceder a la primera posición en una complejidad de $O(1)$. En general, nuestras colas no se beneficiarían de esta estructura de datos.
- **Linked list (sencilla o doblemente enlazada):** Una linked list podría facilitar las operaciones de inserción y borrado de la lista, al no tener que desplazar el resto de elementos.

Con todo, las operaciones de recuperación de 1 elemento en concreto tendrían complejidad $O(n)$, y almacenar todas las direcciones de los nodos aumentaría substancialmente el uso de memoria del contrato.

- **Array estático:** Un array estático tiene el problema de que las inserciones, modificaciones (movimientos), borrados y búsquedas tienen una complejidad de $O(n)$. Con todo, una inserción al final del array, y lo más importante, la obtención de un valor en un índice en concreto tiene una complejidad $O(1)$. Tienen otra ventaja, y es que no es tan intensivo en memoria como otras estructuras más complejas como listas enlazadas o árboles, y además es muy sencillo de implementar. Junto con una estructura auxiliar (mapping) que apunte a cada elemento de la lista se puede bajar la complejidad del caso de uso *getPositionInQueue* a $O(1)$.

Al final se optó por un array estático para la implementación de las colas, junto con una separación de colas por prioridades para paliar las operaciones más costosas. De esta manera tendríamos 1 cola para cada prioridad con sus respectivas operaciones. Cabe destacar en un futuro se podría cambiar la implementación por un Heap, utilizando las direcciones (o identificadores) de los pacientes para el branching, junto con la misma estructura auxiliar de mapping, pero requeriría más investigación.

5.2. Anonimización

Para el aspecto de la anonimización de los pacientes se empezó por dividir el objeto modelo de Patient en 3 objetos:

- **ExtPatient:** El objeto de paciente externo, el cuál solo expone su identificador hashado, el especialista y en la prioridad en la que se encuentra.
- **SensitivePatient:** Objeto que contiene los datos sensibles de un paciente: su dirección de cartera, nombre, y la razón de inserción en la cola.
- **CompletePatient:** El tipo de dato que manejan normalmente los especialistas y la entidad dueña. Es una unión de los dos anteriores objetos, dando a lugar a la información completa, tanto pública como sensible, de un paciente.

Después, se implementó una función de hashing con salt para hashear la dirección de cartera de un paciente y obtener un identificador que será el usado para exponer a los pacientes. Todas las funciones de obtención de datos públicas, es decir, que no sean realizadas por un especialista o la entidad dueña, devuelven los datos anónimos de los pacientes (**ExtPatient**). De esa manera un usuario no autenticado no podrá relacionar fácilmente una dirección de cartera de una persona real con la dirección anónima de un paciente en la cola, garantizando la privacidad.

Junto con ese cambio se implementó una función llamada *getCompletePatientData*, controlada para que solo los especialistas y la entidad dueña puedan usarla. Esta función permite derivar los objetos **CompletePatient** de una lista de direcciones anónimas pacientes, para así poder manejarlos. Se hace la operación en batch pues si no se harían demasiadas llamadas al contrato, una por cada paciente que se quiere deanonimizar, y luego se almacenan los datos en el dispositivo del usuario.

5.3. IPFS

Para el uso de IPFS nuestro contrato tiene algunas limitaciones, concretamente que prácticamente todos los datos tienen que ser validados por el propio contrato y no deben estar

expuestos a modificaciones fuera de él. Esto es, deben ser inmutables incluso para cualquier persona o entidad que no sea el propio contrato. Esto es para garantizar que no se haya cambiado ninguna entrada en los historiales de movimientos y estén sincronizados con la cola actual.

Con todo, conseguimos encontrar un lugar para el uso de IPFS. Una de las razones por las que un paciente puede ser retirado de una lista de espera es por una derivación a un seguro privado. Dicho seguro idealmente debería ser indicado en un comentario de la operación de borrado. Por otro lado, sería interesante saber todos los seguros con los que hay convenios y en qué se especializa cada uno (e.g.: cirugías estéticas, cirugías internas, etc.). Dicha lista no tiene por qué tener su integridad verificada por el contrato ni tampoco guarda dependencias con el contrato.

Entonces lo que hizo es implementar la lista de convenios, que le puede interesar a los usuarios, en IPFS, y guardar el hash del archivo actualizado en el contrato. La única entidad que puede actualizar el hash es la propietaria del contrato, y cualquier usuario o front-end puede recuperarlo para su visualización a través de la propiedad ***conveniosIPFSHash***.

En la siguiente imagen podemos ver el archivo json subido al nodo, este contiene la lista de empresas de seguros a las que se les pudiera referir pacientes gracias al convenio.

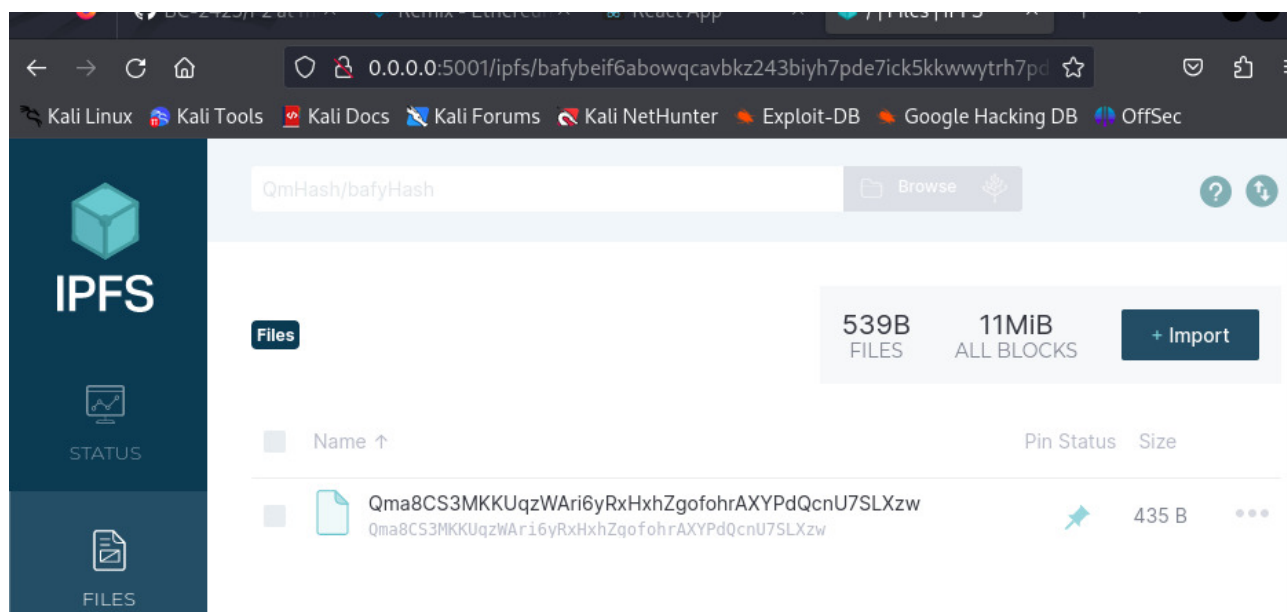


Figura 14: Imagen de fichero de convenios subidos a IPFS.

Se utiliza la función `setConveniosIPFSHash` para actualizar el Hash de IPFS del archivo de convenios, este Hash representa la ubicación del archivo en IPFS.

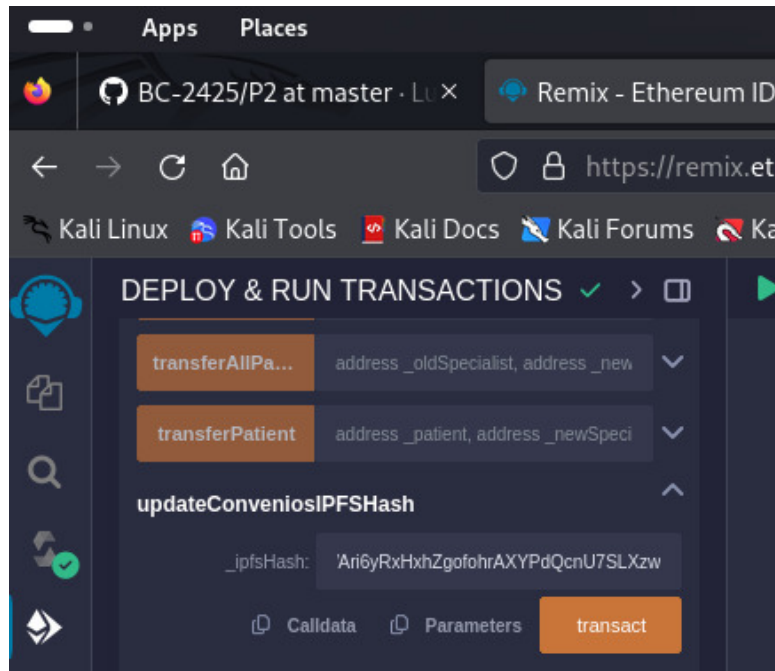


Figura 15: Actualización del Hash IPFS en el contrato.

En la siguiente imagen se utiliza la función `getConveniosIPFSHash`, que permite obtener el Hash de IPFS almacenado en el contrato, de modo que cualquier usuario puede llamar a esta función para consultar el archivo de convenios en IPFS.

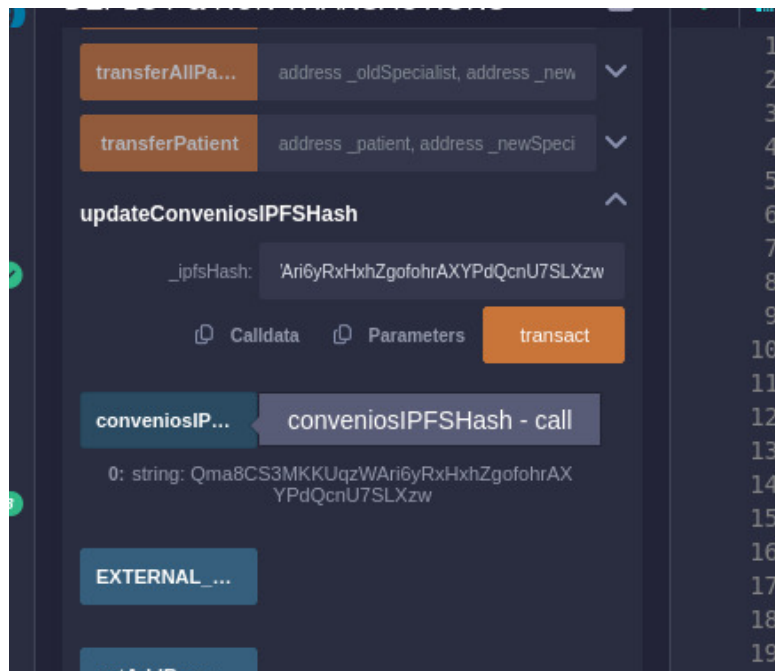


Figura 16: Ejecución de la función para obtener el Hash del fichero de convenios.

5.4. Front-end

El front-end de la aplicación se implementó como una SPA en **Vue**, y se utiliza la librería de **Web3JS** para interactuar con el contrato y la cartera de MetaMask del usuario.

Cuando un usuario conecta su cartera de MetaMask, se autentica con los roles que posee dentro del contrato mediante la función ***getRole()***. Los bits del número obtenido luego se usan para quitar o añadir elementos en las páginas y limitar la navegación del usuario. La navegación se estructura en 4 grandes bloques, uno para cada actor del contrato, donde solo tienen acceso los usuarios que poseen el rol necesario dentro del contrato.

En el proyecto se usan dos stores para almacenar todas las funcionalidades del wallet y el contrato (web3JS), de esa manera se pueden utilizar en diversos componentes y partes del proyecto fácilmente. Cada llamada que se hace al contrato que no vaya a alterar su estado se utiliza el método de ***call()***, manteniendo la usabilidad del contrato sin interrupciones de transacciones. Solo se usa el método de ***send()*** cuando se vaya a realizar una operación que modifique los datos del contrato.

Se implementaron todos los casos de uso principales de la aplicación front-end:

- Manejo de especialistas
- Manejo de pacientes
- Manejo de colas
- Visualización de colas públicas
- Visualización del paciente en la cola
- Visualización de los historiales

Mientras que se dejaron fuera los casos de uso relacionados con la transferencia de pacientes entre especialistas.

La aplicación sigue un diseño fácil e intuitivo, con interfaces claros y responsivos. También está optimizada para su uso en dispositivos móviles.

6. Demostración del funcionamiento del sistema

En seguida se mostrarán las páginas y vistas que dispone el front-end de la aplicación:

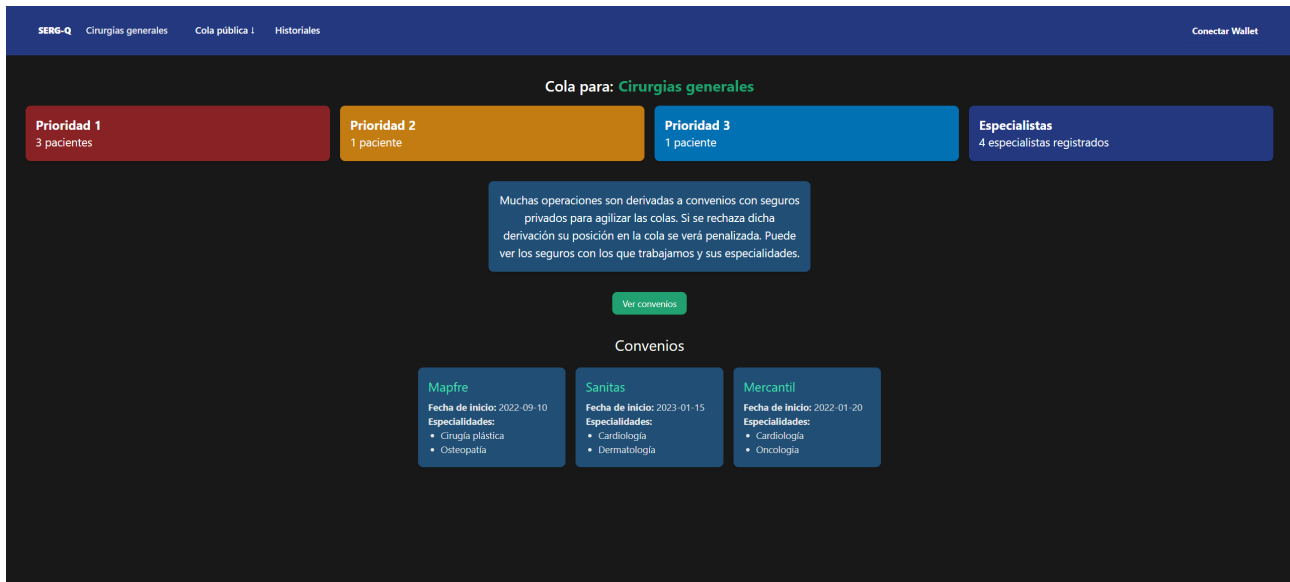


Figura 17: Página principal de la aplicación para usuarios externos, donde se puede ver estadísticas de la plataforma.

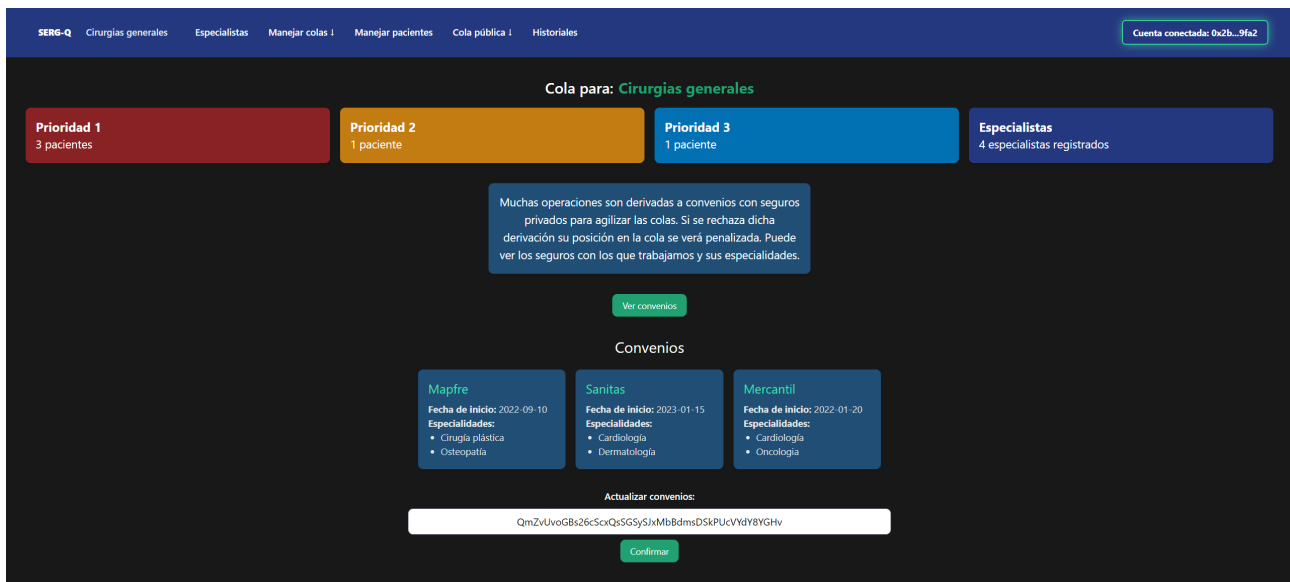


Figura 18: Página principal de la aplicación para la entidad dueña (SERGAS). Puede actualizar el hash del archivo de convenios en el IPFS.

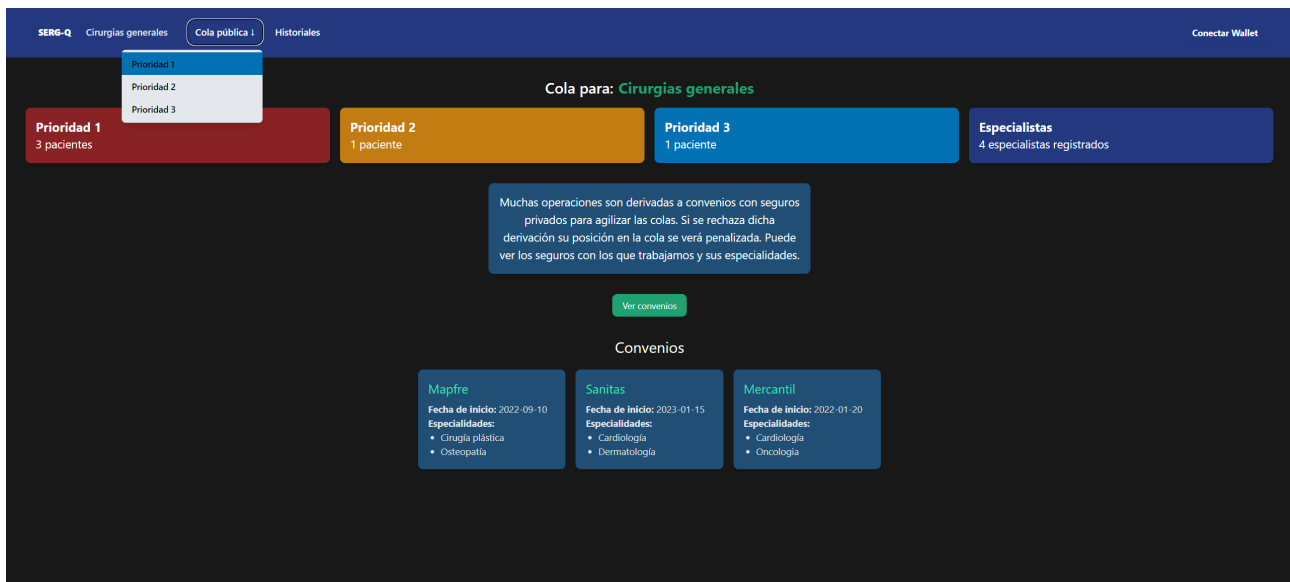


Figura 19: Selector para escoger la cola pública que ver.

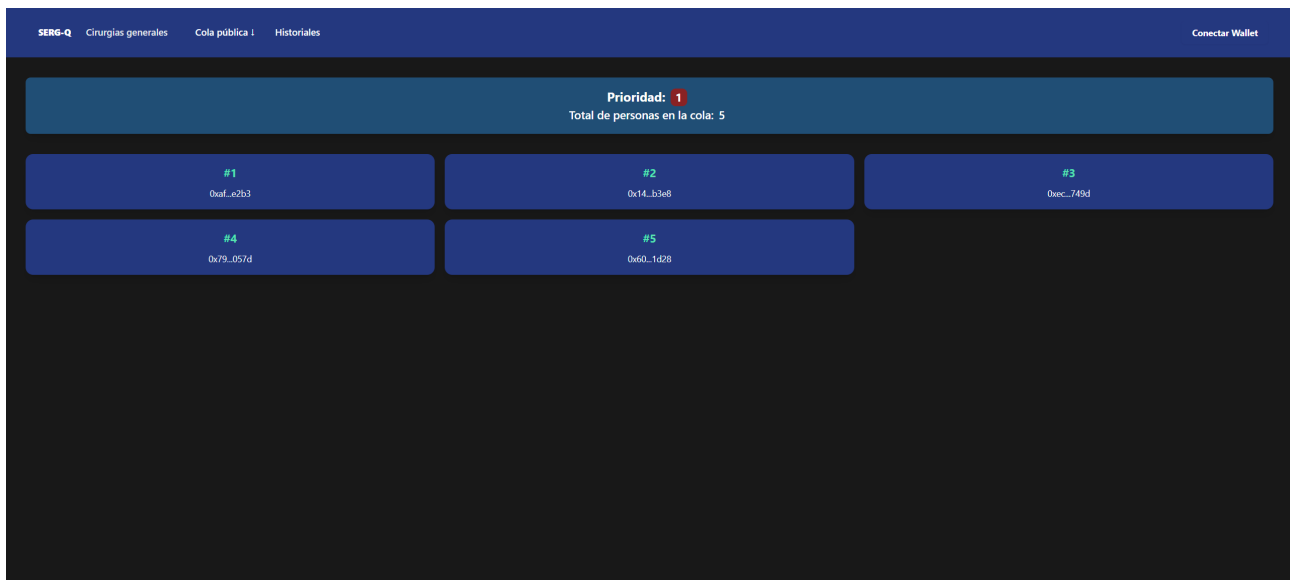


Figura 20: Cola pública para la prioridad 1.

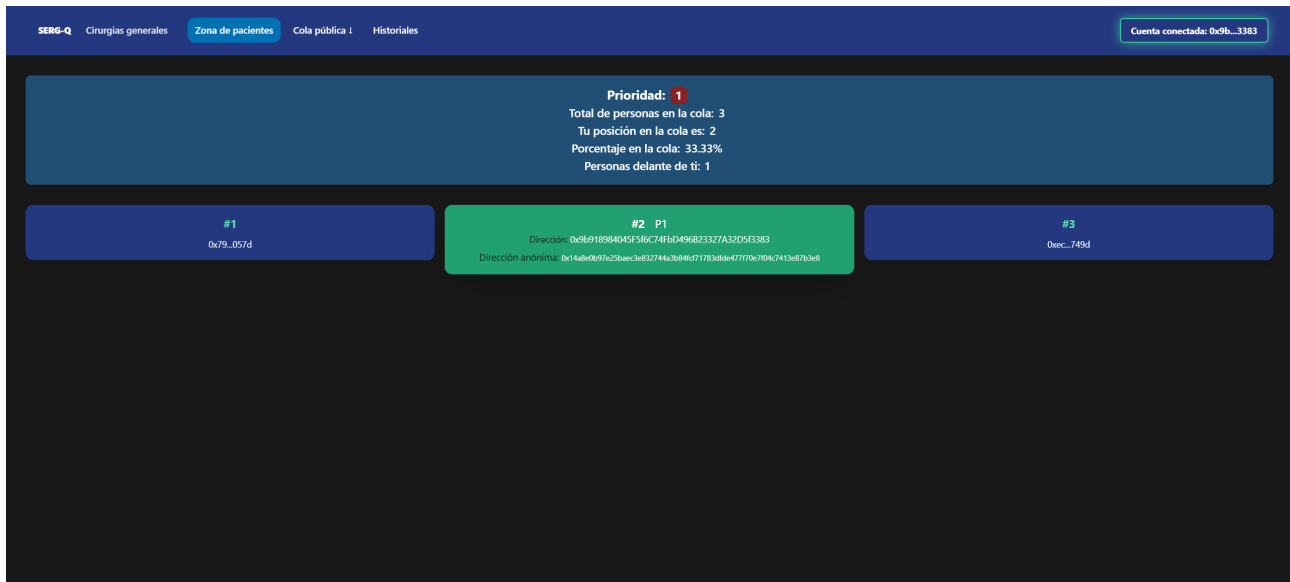


Figura 21: Vista del paciente de su posición en la cola, donde es destacado y se le brinda información extra.

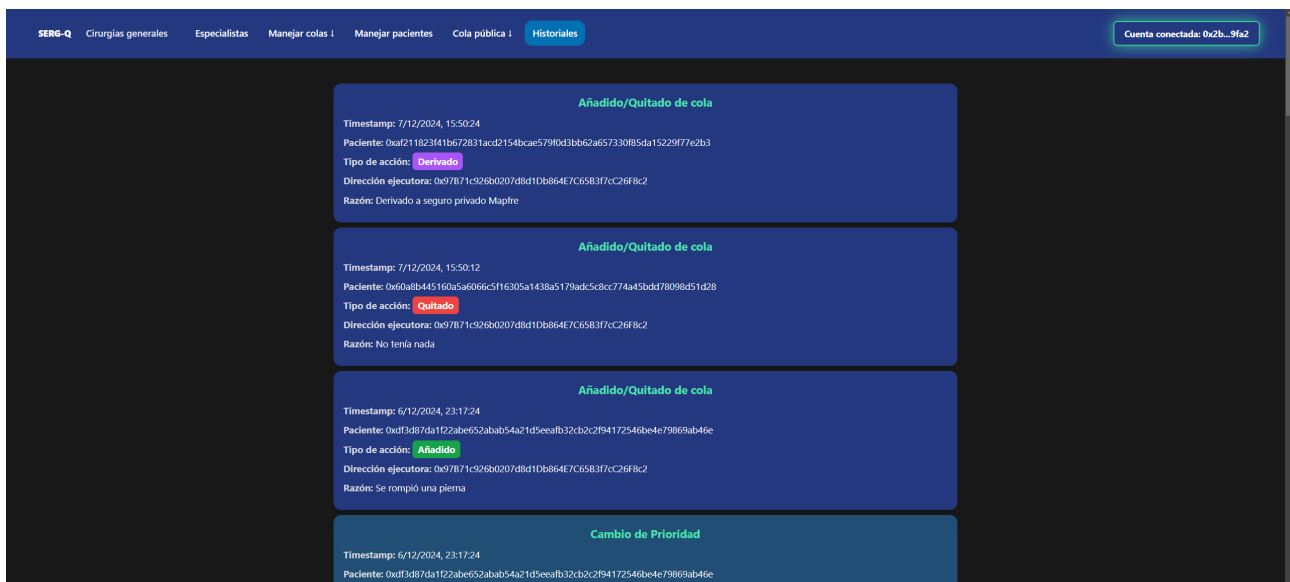


Figura 22: Página de historiales de cambios en las colas.

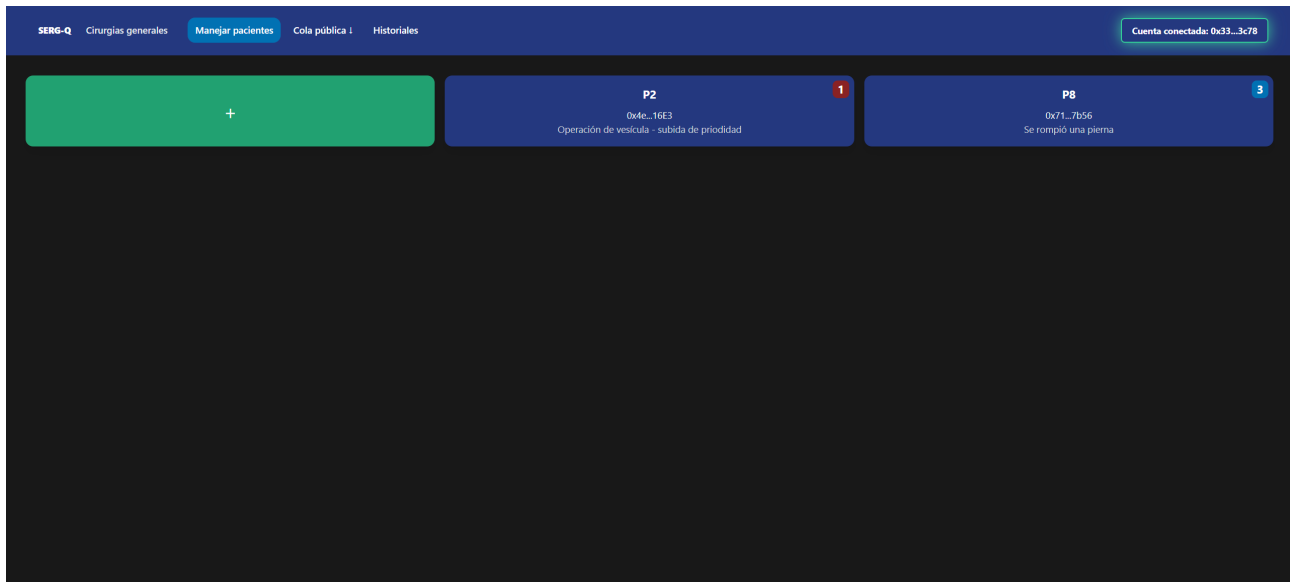


Figura 23: Página del especialista para manejar sus pacientes.

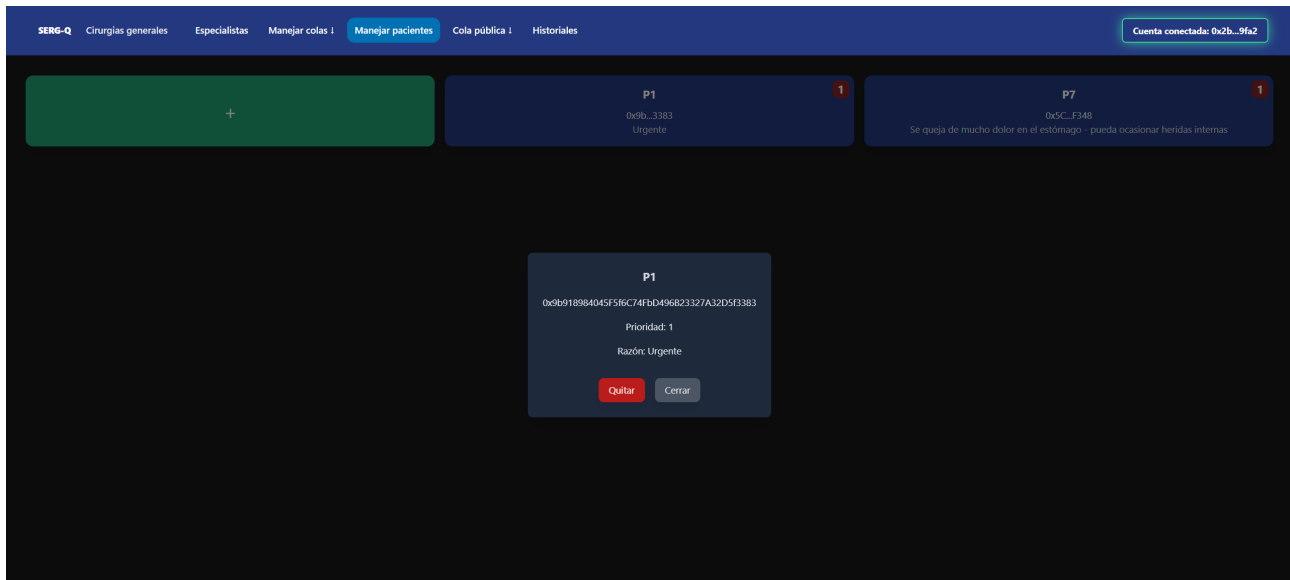


Figura 24: Modal para quitar paciente de la cola siendo un especialista.

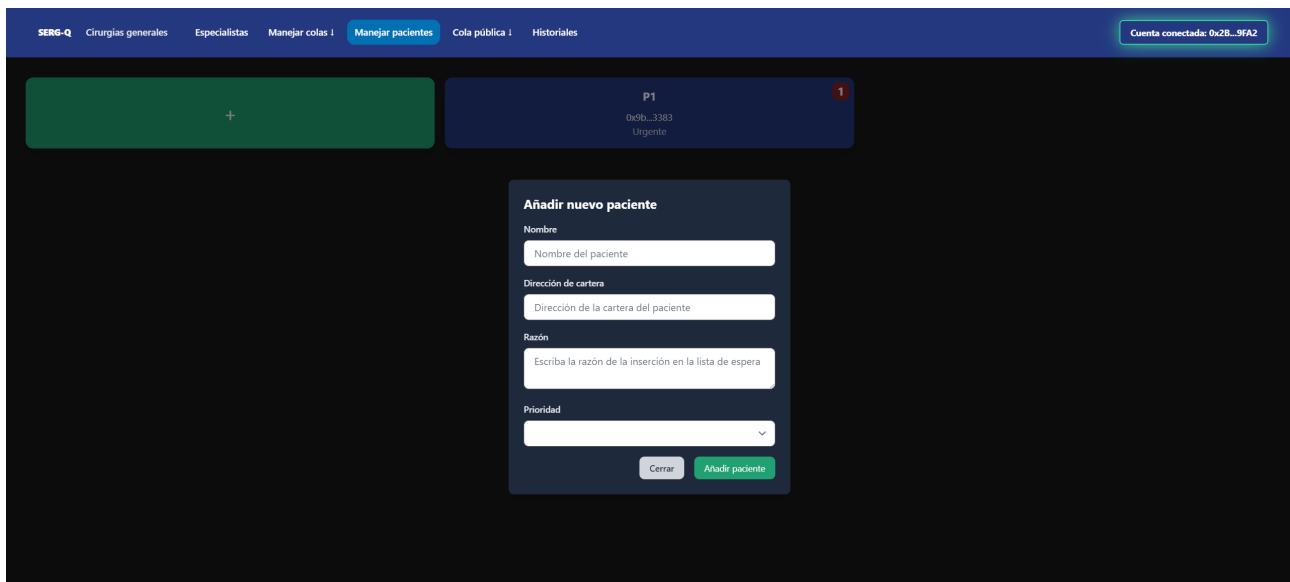


Figura 25: Modal del especialista para añadir un nuevo paciente a la cola.

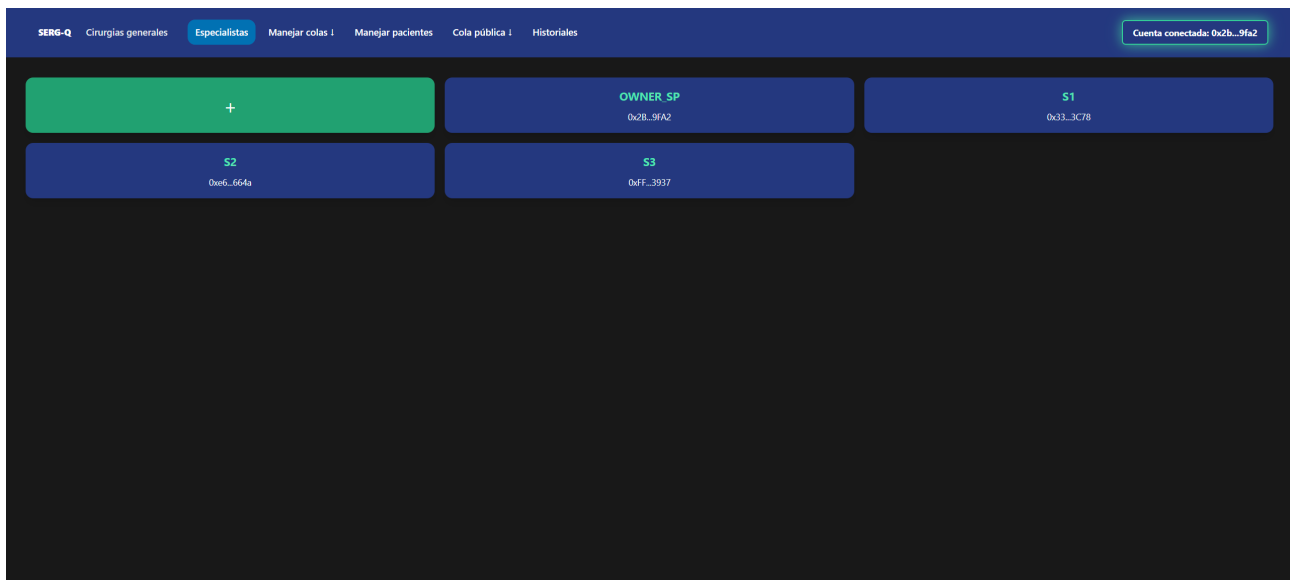


Figura 26: Vista para manejar los especialistas siendo el SERGAS.

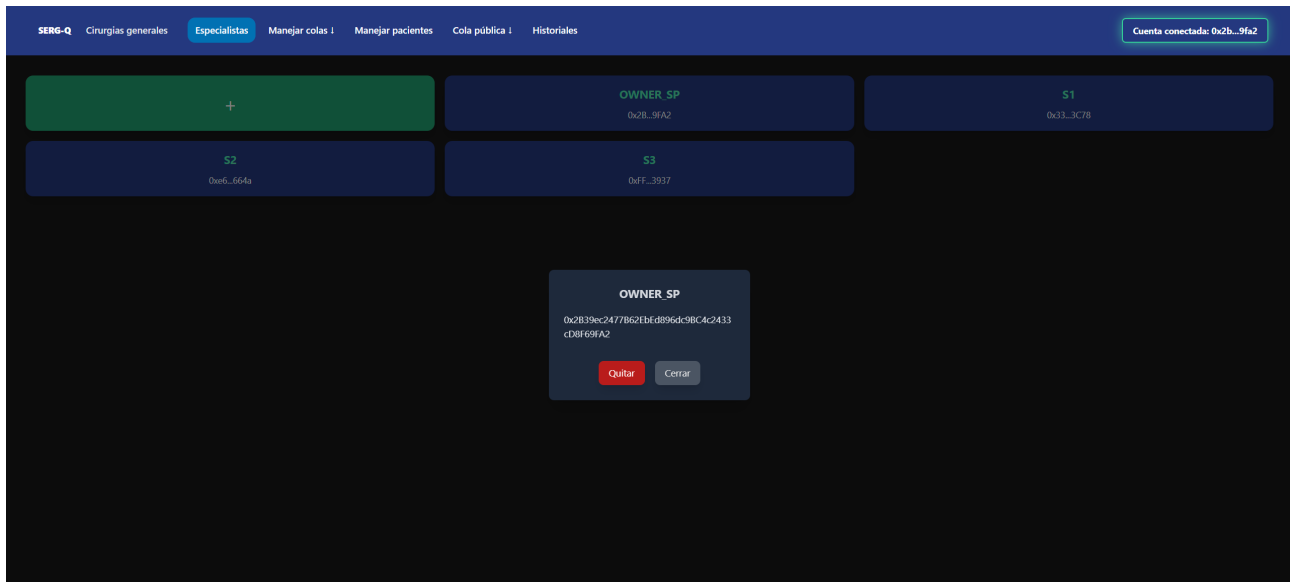


Figura 27: Modal para quitar especialista.

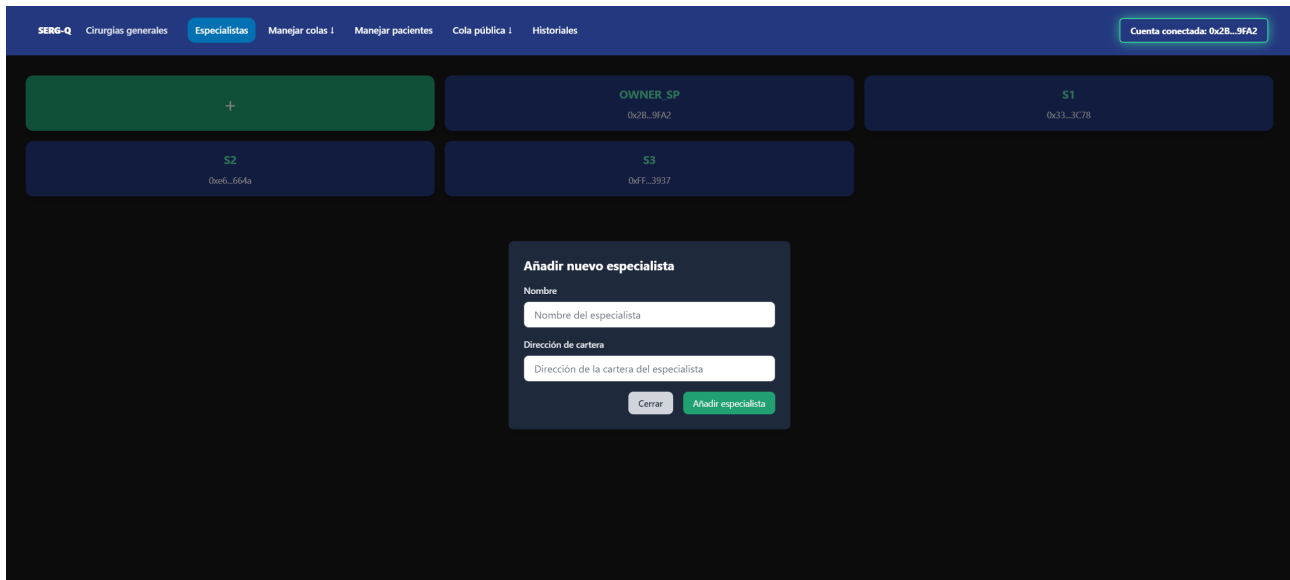


Figura 28: Modal para añadir un especialista.

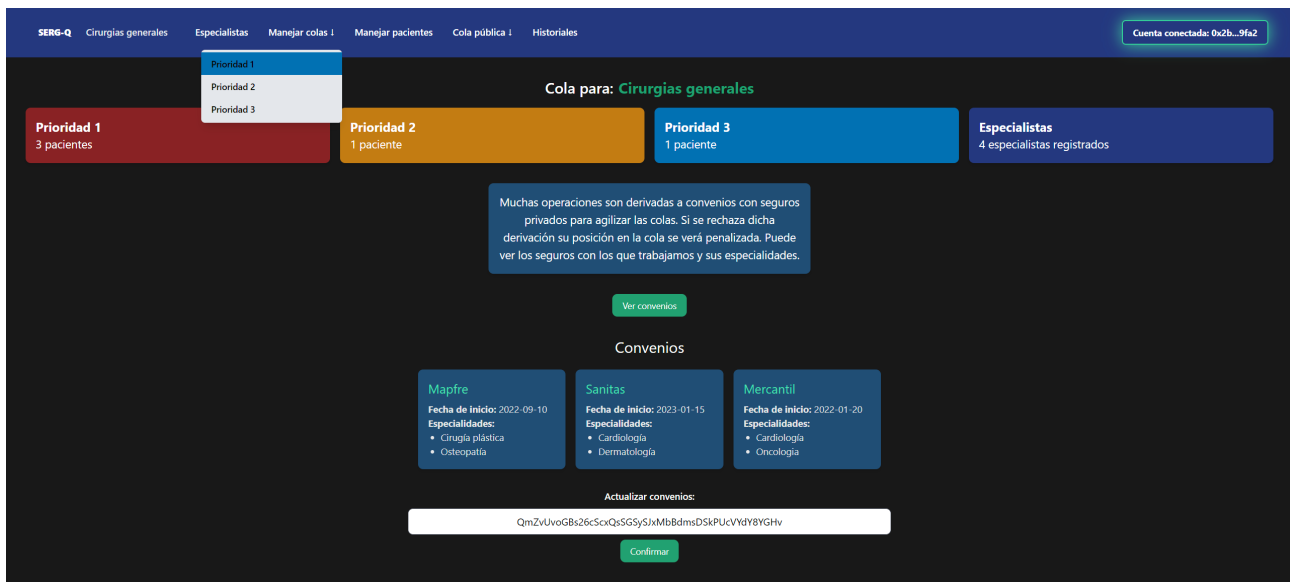


Figura 29: Selector de SERGAS para escoger cola a manejar.

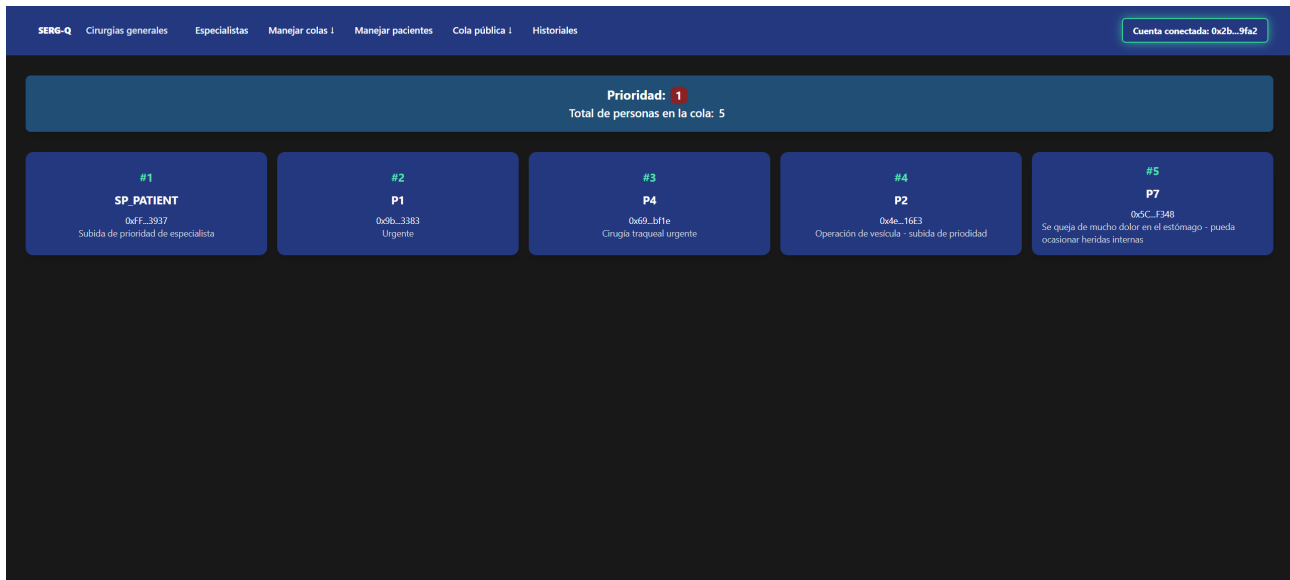


Figura 30: Vista para manejar las colas.

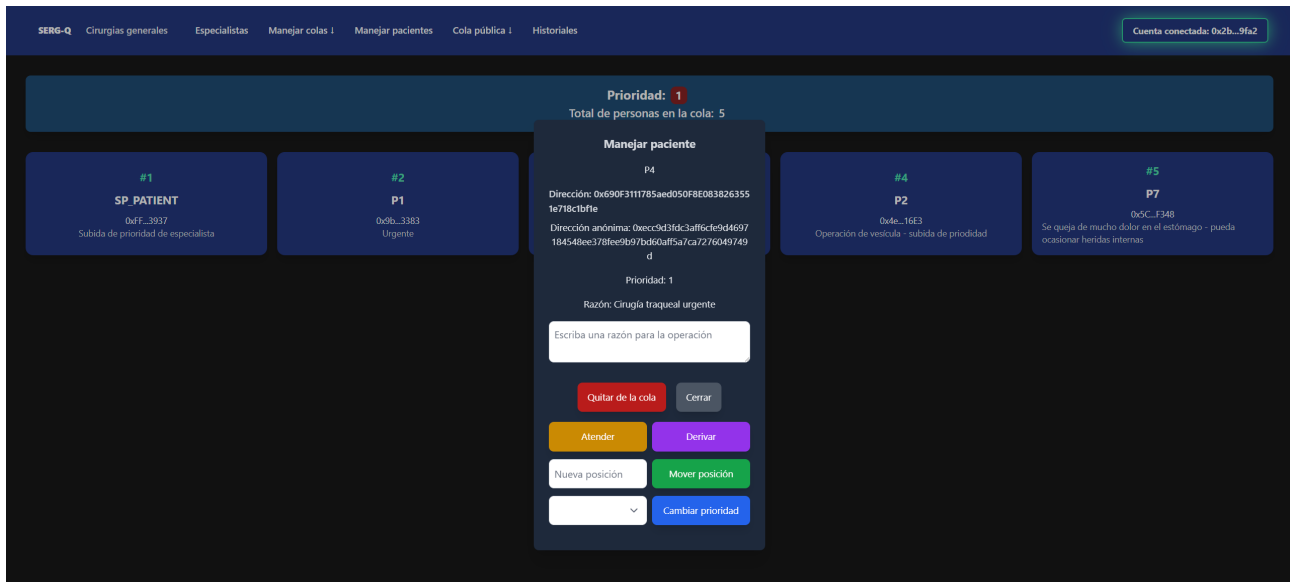


Figura 31: Modal para manejar un paciente en la cola, con todas las acciones posibles.

6.1. Despliegue

Para ejecutar la aplicación es necesario primero tener instalado NodeJS, y tener el smart contract desplegado con su nombre para las colas asignado (por ejemplo, Cirugías generales"). Luego, se pega la dirección del contrato dentro de la variable de entorno correspondiente en el fichero .env del proyecto.

Después, se ejecuta **npm install** dentro de la carpeta principal del proyecto front-end (Tutelado/sergas-queue/) para instalar las dependencias, y luego se ejecuta en entorno de desarrollo con **npm run dev**. Si se quiere ejecutar el entorno de producción se debe compilar con **npm run build** y servir con **npm run preview**.

7. Análisis de los riesgos de seguridad

7.1. Ejemplo: SRM (Security Risk Management)

Activos Identificados

- **Contratos Inteligentes:** Implementan la lógica del sistema de listas.
- **Datos de Pacientes:** Incluyen información sensible como prioridades y especialistas asignados.
- **Infraestructura Blockchain:** Red Ethereum y nodos asociados al sistema.
- **Interfaz de Usuario (Frontend):** Permite la interacción de pacientes, especialistas y auditores.
- **Historiales de Modificaciones:** Registros inmutables de las operaciones realizadas en la lista.

Amenazas y Vulnerabilidades

Amenaza	Activo Impactado	Vulnerabilidad Asociada
T1: Manipulación de datos	Contratos Inteligentes	Errores en la lógica del contrato o funciones mal definidas.
T2: Robo de datos sensibles	Datos de Pacientes	Mecanismos de control de acceso débiles o mal configurados.
T3: Ataques DoS	Infraestructura Blockchain	Sobrecarga de nodos mediante alto consumo de gas.
T4: Desanonimización	Historiales y Datos de Pacientes	Relación entre direcciones públicas y datos personales.
T5: Alteración de historiales	Historiales de Modificaciones	Falta de mecanismos adicionales para garantizar integridad.

Contramedidas Propuestas

1. **Control de Acceso Descentralizado (C#1):** Utilizar permisos granulares gestionados mediante smart contracts para restringir accesos no autorizados.
2. **Cifrado de Datos Sensibles (C#2):** Cifrar datos sensibles off-chain antes de almacenarlos o procesarlos.
3. **Uso de Libro Mayor Inmutable (C#3):** Almacenar historiales en la blockchain para garantizar su integridad y trazabilidad.
4. **Anonimización de Datos (C#4):** Utilizar hash con salt únicos para anonimizar direcciones públicas de pacientes.
5. **Mitigación de Ataques DoS (C#5):** Implementar mecanismos de límite de gas y validaciones previas para evitar la sobrecarga de nodos.
6. **Monitorización y Auditoría de Contratos (C#6):** Realizar auditorías periódicas y establecer recompensas tipo *bug bounty* para la detección temprana de vulnerabilidades.
7. **Cifrado de Comunicaciones (C#7):** Asegurar todas las comunicaciones cliente-servidor mediante TLS y proteger interacciones con APIs.

Impacto y Probabilidad de Riesgos

Riesgo	Impacto	Probabilidad	Nivel de Riesgo
Manipulación de datos	Alto	Media	Alto
Robo de datos sensibles	Alto	Alta	Crítico
Ataques DoS	Medio	Media	Medio
Desanonimización	Alto	Media	Alto
Alteración de historiales	Alto	Baja	Medio

8. Plan de pruebas

En una etapa inicial, el sistema será probado con colas no prioritarias y un grupo reducido de personas. Este enfoque busca simular un entorno controlado que permita evaluar la funcionalidad básica del sistema sin sobrecargarlo con casos de uso complejos o grandes volúmenes de datos. Estas pruebas iniciales se enfocarán en verificar la correcta implementación de

las operaciones básicas (agregar, eliminar y mover pacientes dentro de la cola) y en detectar posibles errores que podrían surgir en los procesos fundamentales del sistema.

Resultados Esperados

- Validación Funcional: Se espera que el sistema ejecute todas las operaciones básicas sin errores y que los datos almacenados en la blockchain sean consistentes y verificables.
- Corrección de Errores Iniciales: Durante esta etapa, es probable que se identifiquen errores menores en la lógica de los contratos inteligentes o en la interacción con la interfaz, los cuales deberán ser corregidos antes de avanzar a fases más complejas.
- Punto de comparación inicial: Los resultados de esta etapa inicial servirán como referencia para validar futuras iteraciones del sistema, permitiendo la comparación del rendimiento y la funcionalidad en escenarios más avanzados.

9. Lean Canvas

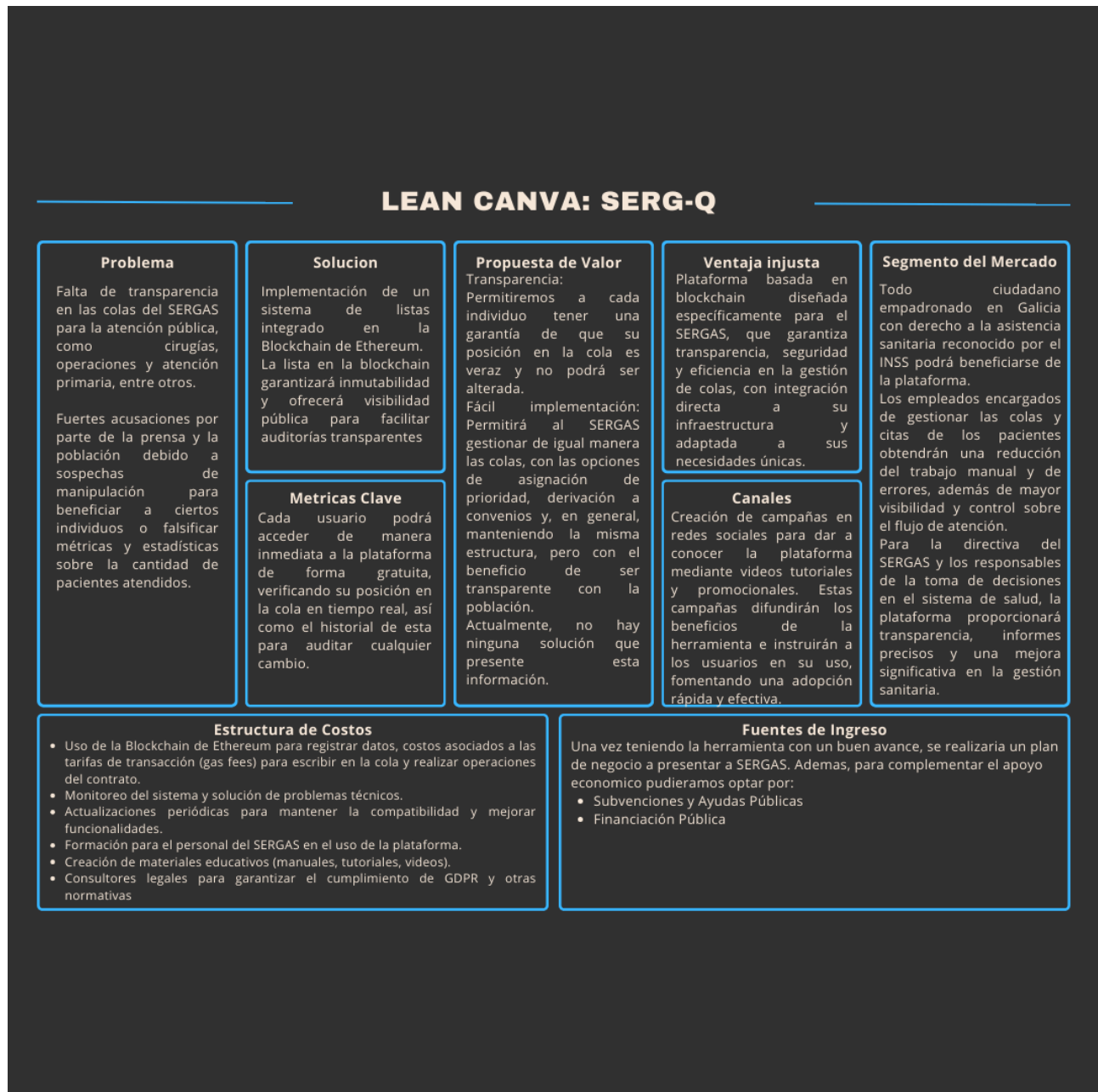


Figura 32: LEAN CANVA SERG-Q.

10. Aspectos legales

En cuanto a los aspecto legales debemos garantizar que los datos personales cumplan con el GDPR, específicamente con la anonimización de estos.

La anonimización según el GDPR establece que los datos personales deben ser transformados de manera irreversible, anulando cualquier posibilidad de identificar de manera directa o indirecta a una persona física, si realizamos este proceso los datos dejan de considerarse personales y estaríamos cumpliendo con el GDPR, además pudiéramos pedir apoyo de SERGAS ya que al ser la empresa de salud publica de Galicia ha de tener previa experiencia extensa en este apartado.

10.1. Estructura del proyecto

Dentro del smart contract el proyecto se organiza de la siguiente manera:

- **ControlQueue.sol**: Fichero de contrato principal, el que se compila y se expone.
- **QueueList.sol**: Fichero que implementa los métodos de manejo de colas e historiales. Lo implementa el ControlQueue.sol.
- Carpeta de **roles/**: Dentro se implementan los contratos para el role checking dentro del contrato
- Carpeta de **models/**: Dentro se implementan los modelos y entidades usados por el contrato
 - **Patient.sol**
 - **Priority.sol**
 - **Specialis.sol**

En el front-end se implementa una arquitectura clásica de frameworks modernos de JS/TS, organizado en componentes y views:

- **.env**: se guardan las variables de entorno importantes para el funcionamiento del programa como la dirección del smart contract.
- **src**:
 - **assets/**: Dentro contiene CSS y un logo.
 - **components**: Dentro se guardan todos los componentes reutilizables de Vue. A su vez también tiene jerarquías de componentes para owner, specialist y patient.
 - **models/**: Dentro de guardan los modelos de datos que se manejan como tipos (p.ej.: patient, specialist, convenio...) y el ABI del contrato.
 - **router/**: Dentro tienen un fichero que indica las rutas que tendrá la aplicación y la lógica de comprobación de roles para acceder a esas rutas.
 - **stores/**: Dentro se guardan las stores que contiene la lógica principal de interacción con el contrato (en **web3.ts**) y con el wallet de MetaMask (**wallet.ts**).
 - **views/**: Dentro contiene las páginas template para las principales rutas de la aplicación, y están organizadas por jerarquías como los componentes: owner, specialist, patient, y las públicas que no van en ninguna carpeta.
 -
 - **utils.ts**: Contiene funciones de mapeado de calldata del contrato a tipos definidos en models/ y funciones de auxilio como por ejemplo para acortar direcciones.
 - **App.vue** y **main.ts**: Los main entry points de la aplicación.

11. Conclusiones

El desarrollo de un sistema de listas de espera basado en blockchain para SERGAS presenta un avance significativo en la digitalización a las necesidades actuales para los servicios de salud publica. Gracias a este trabajo pudimos demostrar como la aplicación de blockchain

en procesos cotidianos permite mejorar el funcionamiento, transparencia, descentralización e inmutabilidad comparado a su aplicación actual.

En general podemos definir las conclusiones generales del proyecto de la siguiente manera:

Innovación en la Gestión de Listas de Espera:

- La implementación de blockchain permite garantizar la trazabilidad de todas las operaciones en las listas de espera, eliminando cualquier sospecha de manipulación y ofreciendo una visión clara de los movimientos realizados.
- La visibilidad en tiempo real de la posición en la cola para los pacientes representa un salto hacia una experiencia más transparente y confiable.

Eficiencia y Automatización:

- La automatización de las operaciones mediante contratos inteligentes reduce significativamente la dependencia de procesos manuales y el riesgo de errores humanos.
- La integración de registros inmutables y accesibles asegura que los historiales de modificaciones sean verificables y estén disponibles para auditorías en cualquier momento.

Ventajas de la Descentralización:

- Al eliminar los puntos únicos de fallo inherentes a los sistemas centralizados, la red descentralizada garantiza la disponibilidad del sistema incluso en situaciones de fallo técnico o ataques externos.
- La capacidad de operar de manera autónoma, sin necesidad de intermediarios, aumenta la confiabilidad y eficiencia del sistema.

Escalabilidad y Adaptabilidad:

- El modelo propuesto podrá ser aplicable en otros contextos, como la gestión de historias clínicas, suministro de medicamentos o asignación de citas médicas.

Desafíos:

- A pesar de los avances logrados, el trabajo identifica desafíos técnicos y operativos, como la necesidad de optimizar el consumo de gas en la blockchain y garantizar la escalabilidad para manejar grandes volúmenes de usuarios.

En conclusión, este proyecto no solo ha demostrado la viabilidad técnica de implementar blockchain en la gestión de listas de espera, sino que también permite abrir la mente y ver una implementación más amplia en la administración de servicios públicos. Al ofrecer un sistema más transparente, seguro y eficiente, esta solución puede convertirse en un modelo a seguir para otras instituciones que buscan modernizar sus procesos y aumentar la confianza de sus usuarios. La tecnología blockchain, aplicada con un enfoque práctico y tiene el potencial de mejorar significativamente la calidad y accesibilidad de los servicios públicos en el futuro cercano.

12. Líneas futuras

Dentro del alcance actual de la aplicación quedarían algunas cuestiones por mejorar.

Dentro de lo que es funcionamiento del smart contract en sí:

- Optimizar las estructuras de datos para las colas (como mencionado anteriormente).

- Implementar la obtención parcial de la cola e historiales para implementar paginación en el front-end.
- No utilizar el mismo salt para todos los hashings, si no crear uno para cada paciente y almacenarlo de manera segura y que no se pueda obtener.
- Para evitar la pérdida de datos sensibles, añadir otro parámetro a las llamadas que los devuelve (por ejemplo, *getCompletePatientData*) que sea un nonce firmado por el usuario. En el contrato luego se verifica el propietario de la firma para asegurarse de que es realmente quien dice ser. Esto en concreto solo afecta a las llamadas **call()** desde el frontend, y actuaría como barrera ante la suplantación en dichas llamadas, ya que, a contrario de **send()**, estas no van firmadas.

Luego, en el front-end se podrían hacer ciertas mejoras para una mejor experiencia de usuario:

- Añadir los casos de uso de transferencia de pacientes entre especialistas.
- Mostrar una lista de todos los especialistas en la plataforma.
- Poder filtrar y ordenar todas las listas e historiales.
- Mostrar los datos de la persona autenticada explícitamente (Nombre, roles...).
- Paginación en las colas e historiales para no cargar cantidades enormes de datos sin necesidad.
- División de los historiales por tipos en páginas para una mejor visualización.
- Mejorar los colores para la accesibilidad y visibilidad

Tras estas mejoras y lograr una aceptación amplia de nuestra herramienta, sería ideal poder ampliar este sistema y adaptarlo a otros servicios del sistema de salud dentro de SERGAS como en otras instituciones que gestionan estos aspectos dentro de España.

Podemos aprovechar la flexibilidad de la tecnología para implementarlo en otros aspectos como lo pudieran ser:

- Gestión de historias clínicas digitales
- Monitoreo, rastreo y control de medicamentos y suministros.
- Gestión hospitalaria.
- Gestión de prescripciones medicas.

13. Lecciones aprendidas

Por lo general, un smart contract se puede implementar como un backend remoto para aplicaciones front-end. El aspecto de la inmutabilidad y no depender de ninguna entidad que lo hostee pueden ser interesantes para muchas aplicaciones.

13.1. Incidencias

La implementación de IPFS fue un desafío, ya que no conseguíamos encontrar un caso de uso que se ajustase. En nuestro escenario queremos garantizar la máxima integridad e inmutabilidad posible. Además, nuestro contrato depende y trabaja directamente con esos datos,

y hasta donde sabemos un contrato no puede acceder o interactuar directamente con el IPFS. En el supuesto caso de que se pudiese, podría ser una buena idea hacer 'dumps' del historial, guardando el hash íntegro en el contrato (de manera que nadie más pudiese modificarlo), para que así no se guardase toda esa información dentro de la blockchain. En nuestro caso aplicarlo sobre los historiales sería de mayor interés, ya que pueden crecer indefinidamente.

14. Tiempo dedicado/esfuerzo

Luis Eduardo Gómez Moran : 48h

Luca D'angelo Sabin : 48h

Referencias

- [1] Agencia Estatal del Estado. Boe nº 134. <https://www.boe.es/buscar/act.php?id=BOE-A-2003-11266>.
- [2] SERGAS. Información al paciente sobre listas de espera. <https://www.sergas.es/Asistencia-sanitaria/Que-son-os-niveis-de-prioridade-dunha-intervenci%C3%B3n-cir%C3%B3nica>.
- [3] SERGAS. Información sobre los niveles de prioridad de intervención quirúrgica. <https://www.sergas.es/Asistencia-sanitaria/Que-son-os-niveis-de-prioridade-dunha-intervenci%C3%B3n-cir%C3%B3nica?idioma=es>.